Kamesh S
192521358
DWDM

LAB EXPERIMENTS

R PROGRAMMING

EXPERIMENT-1:
To prove the program for addition using R-tool.

Program:

```
a <- 10
b <- 20

sum <- a + b

print(sum)
```

Output Screenshot:

```
> print("Hello World")
[1] "Hello World"
> 10+20
[1] 30
> a <- 10
> b <- 20
>
> sum <- a + b
>
> print(sum)
[1] 30
>
```

EXPERIMENT-2:
To prove the program for subtraction using R-tool.
Program:

Program to subtract two numbers

```
a <- 30
b <- 10

difference <- a - b

print("First number:")
print(a)

print("Second number:")
print(b)

print("Difference:")
print(difference)
```

OUTPUT SCREENSHOT:

```
>
> print("First number:")
[1] "First number:"
> print(a)
[1] 30
>
> print("Second number:")
[1] "Second number:"
> print(b)
[1] 10
>
> print("Difference:")
[1] "Difference:"
> print(difference)
[1] 20
```

EXPERIMENT-3

To prove the program for multiplication using R-tool.
Program:

Program to multiply two numbers

```
a <- 10
b <- 20

product <- a * b

print("First number:")
print(a)

print("Second number:")
print(b)

print("Product:")
print(product)
```

Output Screenshot:

```
> print("First number:")
[1] "First number:"
> print(a)
[1] 10
>
> print("Second number:")
[1] "Second number:"
> print(b)
[1] 20
>
> print("Product:")
[1] "Product:"
> print(product)
[1] 200
>
```

EXPERIMENT-4

To prove the program for division using R-tool.

Program:

```
# Program to divide two numbers

a <- 20
b <- 5
```

```
quotient <- a / b

print("First number:")
print(a)

print("Second number:")
print(b)

print("Quotient:")
print(quotient)
```

Output Screenshot:

```
> print("First number:")
[1] "First number:"
> print(a)
[1] 20
>
> print("Second number:")
[1] "Second number:"
> print(b)
[1] 5
>
> print("Quotient:")
[1] "Quotient:"
> print(quotient)
[1] 4
>
```

EXPERIMENT-5

To write the program for odd or even using R-tool.

Program:

```
# Program to check odd or even

num <- 10

if (num %% 2 == 0) {
  print("The number is Even")
} else {
```

```
  print("The number is Odd")
}
```

OUTPUT SCREENSHOT:

```
[1] "The number is Even"
> |
```

EXPERIMENT-6

To write the program for mean,median,mode.

Program:

```
# Program to calculate Mean, Median and Mode

numbers <- c(10, 20, 20, 30, 40)

# Calculate Mean
mean_value <- mean(numbers)

# Calculate Median
median_value <- median(numbers)

# Calculate Mode
mode_value <- as.numeric(names(
  table(numbers)[table(numbers) == max(table(numbers))]
))

# Display the results
print("Numbers:")
print(numbers)

print("Mean:")
print(mean_value)

print("Median:")
```

```
print(median_value)

print("Mode:")
print(mode_value)
```

Output Screenshot:

```
[1] 10 20 20 30 40
>
> print("Mean:")
[1] "Mean:"
> print(mean_value)
[1] 24
>
> print("Median:")
[1] "Median:"
> print(median_value)
[1] 20
>
> print("Mode:")
[1] "Mode:"
> print(mode_value)
[1] 20
```

EXPERIMENT-7

To write the program for summary using R-tool.

Program:

```
# Create vectors
names <- c("siri", "mahi", "chiru")
age <- c(23, 24, 25)
marks <- c(88, 78, 25)

# Create a data frame
df <- data.frame(names, age, marks)

# Display summary of age
summary(df$age)

# Display the complete data frame
```

```
print(df)

# Write data frame to CSV file
write.csv(df, "datafr.csv", row.names = FALSE)
```

OUTPUT SCREENSHOT:

```
> # Display summary of age
> summary(df$age)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   23.0    23.5    24.0    24.0    24.5    25.0
```

EXPERIMENT-8

To write the program for the greatest among three numbers.

Program:

```
# Program to find the greatest among three numbers

a <- 10
b <- 25
c <- 15

if (a >= b && a >= c) {
  print("The greatest number is:")
  print(a)
} else if (b >= a && b >= c) {
  print("The greatest number is:")
  print(b)
} else {
  print("The greatest number is:")
  print(c)
}
```

OUTPUT SCREENSHOT:

```
+ }
[1] "The greatest number is:"
[1] 25
```

EXPERIMENT-9

To write the program for central tendency and data dispersion measures using R tool.

PROGRAM:

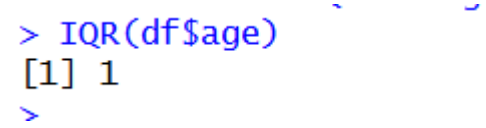
```
names <- c("siri", "mahi", "chiru")
age <- c(23, 24, 25)
marks <- c(88, 78, 25)

df <- data.frame(names, age, marks)

# Calculate IQR of age
IQR(df$age)

# Write data frame to CSV file
write.csv(df, "datafr.csv")
```

OUTPUT SCREENSHOT:



EXPERIMENT-10
QUANTILE:
To write the program for central tendency and data dispersion measures.
Program:

```
# Measures of Central Tendency and Data Dispersion

data <- c(10, 20, 20, 30, 40)
```

```
# Central Tendency
mean(data)
median(data)

# Mode
mode_value <- as.numeric(names(which.max(table(data))))
print(mode_value)

# Data Dispersion
range(data)
var(data)
sd(data)
IQR(data)

# Quantiles
quantile(data)
```

OUTPUT SCREENSHOT:

```
> # Quantiles
> quantile(data)
  0%  25%  50%  75% 100%
  10   20   20   30   40
```

EXPERIMENT-11

MID RANGE:

To write the program for central tendency and data dispersion measures.

PROGRAM:

```
# Measures of Central Tendency and Data Dispersion

data <- c(10, 20, 20, 30, 40)

# Central Tendency
mean(data)
median(data)

# Mode
```

```
mode_value <- as.numeric(names(which.max(table(data))))
print(mode_value)

# Mid-Range
mid_range <- (min(data) + max(data)) / 2
print(mid_range)

# Data Dispersion
range(data)
var(data)
sd(data)
IQR(data)
```

OUTPUT SCREENSHOT:

```
> range(data)
[1] 10 40
```

EXPERIMENT-12
Z-SCOORE NORMALIZATION:
To write the program for Z-scoore normalization using R-tool.
Program:

```
# Z-Score Normalization

data <- c(10, 20, 30, 40, 50)

mean_value <- mean(data)
sd_value <- sd(data)

z_score <- (data - mean_value) / sd_value

print("Original Data:")
print(data)

print("Z-Score Normalized Data:")
print(round(z_score, 2))
```

OUTPUT SCREENSHOT:

```
> print("Original Data:")
[1] "Original Data:"
> print(data)
[1] 10 20 30 40 50
>
> print("Z-Score Normalized Data:")
[1] "Z-Score Normalized Data:"
> print(round(z_score, 2))
[1] -1.26 -0.63  0.00  0.63  1.26
```

EXPERIMENT-13

MIN,MAX,MEAN,MINMAX:

To write the program for the minimum,maximum,mean and minmax using r-TOOL

PROGRAM:

```
# Minimum, Maximum, Mean and Min-Max Normalization

data <- c(10, 20, 30, 40, 50)

# Minimum Value
print(min(data))

# Maximum Value
print(max(data))

# Mean Value
print(mean(data))

# Min-Max Normalization
min_value <- min(data)
max_value <- max(data)

normalized_data <- (data - min_value) / (max_value - min_value)

print("Min-Max Normalized Data:")
print(round(normalized_data, 2))
```

OUTPUT SCREEBSHOT:

```
> # Minimum Value
> print(min(data))
[1] 10
>
> # Maximum Value
> print(max(data))
[1] 50
>
> # Mean Value
> print(mean(data))
[1] 30
>
```

```
> # Min-Max Normalization
> min_value <- min(data)
> max_value <- max(data)
>
> normalized_data <- (data - min_value) / (m
>
> print("Min-Max Normalized Data:")
[1] "Min-Max Normalized Data:"
> print(round(normalized_data, 2))
[1] 0.00 0.25 0.50 0.75 1.00
```

EXPERIMENT-14

BAR PLOT AND HORIZONTAL BAR:

To draw the bar plot and horizontal bar using R-tool.

PROGRAM:

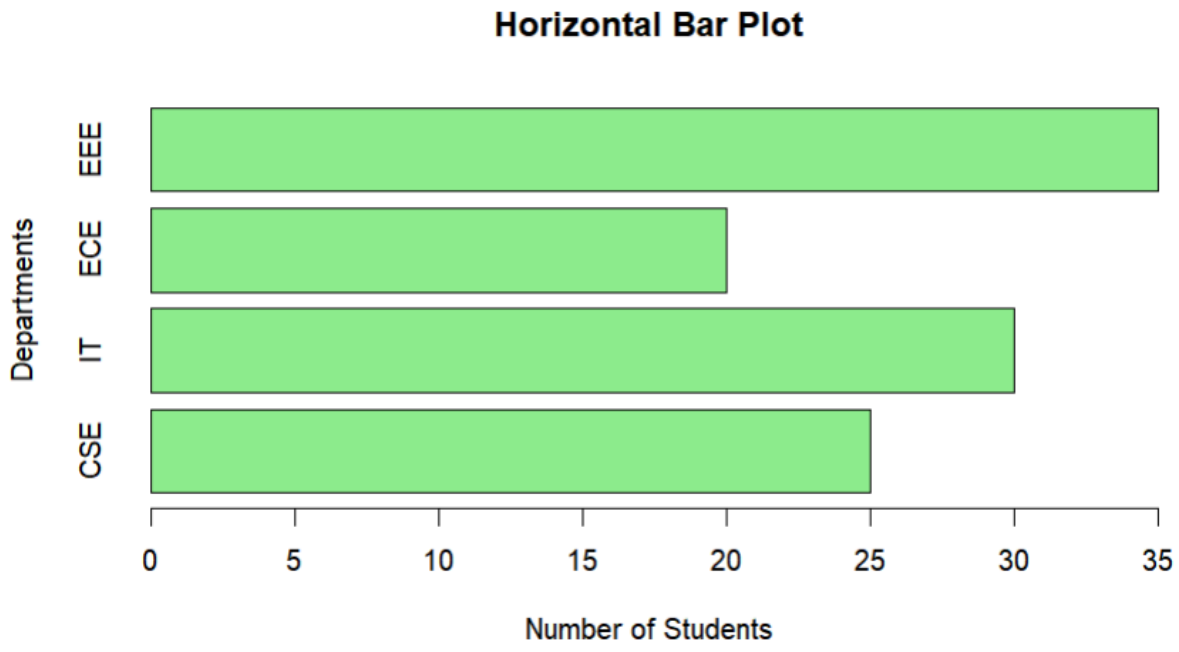
```
# Bar Plot and Horizontal Bar Plot

# Create Data
students <- c(25, 30, 20, 35)
names <- c("CSE", "IT", "ECE", "EEE")

# Vertical Bar Plot
barplot(students,
        names.arg = names,
        main = "Department-wise Students",
        xlab = "Departments",
        ylab = "Number of Students",
        col = "skyblue")
```

```
# Horizontal Bar Plot
barplot(students,
        names.arg = names,
        main = "Horizontal Bar Plot",
        xlab = "Number of Students",
        ylab = "Departments",
        col = "lightgreen",
        horiz = TRUE)
```

OUTPUT SCREENSHOT:



EXPERIMENT-15
BOX PLOT
To draw the box plot using R-tool.

PROGRAM:

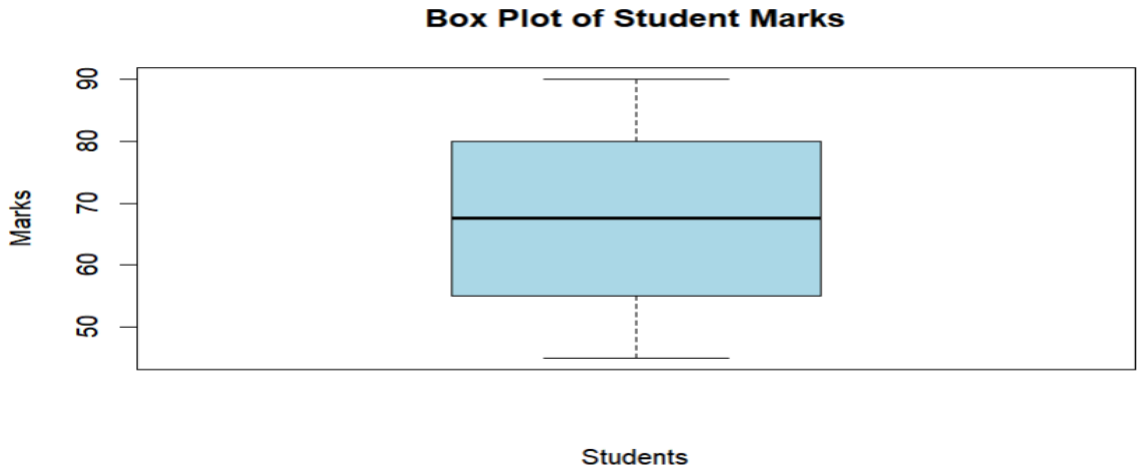
```
# Box Plot

marks <- c(45, 50, 55, 60, 65, 70, 75, 80, 85, 90)

boxplot(marks,
```

```
    main = "Box Plot of Student Marks",
    xlab = "Students",
    ylab = "Marks",
    col = "lightblue",
    border = "black")
```

OUTPUT SCREENSHOT:



EXPERIMENT-16

HISTOGRAM:

To draw the histogram plot using R-tooll.

PROGRAM:

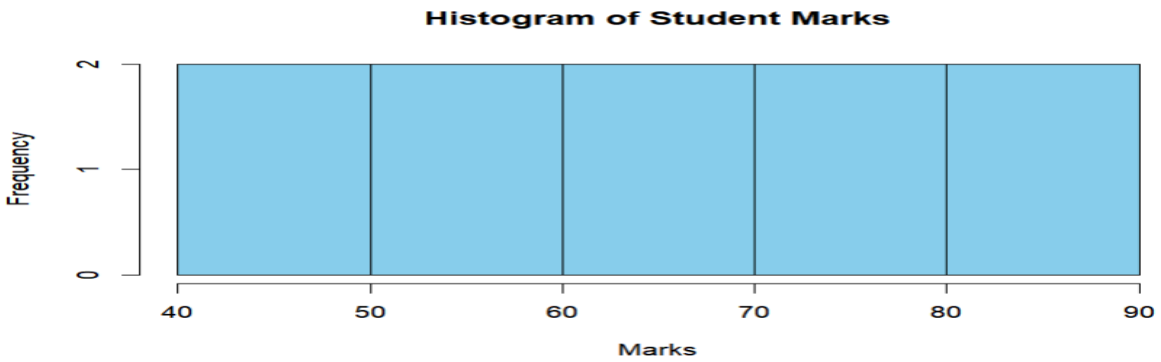
```
# Histogram Plot

marks <- c(45, 50, 55, 60, 65, 70, 75, 80, 85, 90)

hist(marks,
    main = "Histogram of Student Marks",
    xlab = "Marks",
    ylab = "Frequency",
    col = "skyblue",
```

```
    border = "black")
```

OUTPUT SCREENSHOT:



EXPERIMENT-17

CORRELATION ANALYSIS:

To write the program for correlation analysis using R-tool.

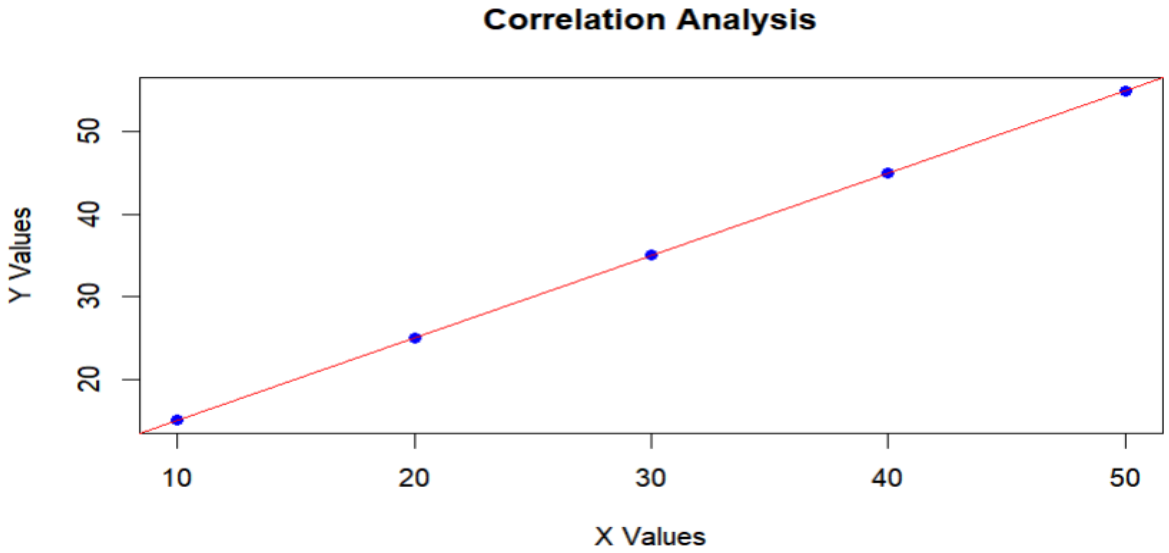
PROGRAM:

```
# Correlation Analysis
# Create Data
x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 35, 45, 55)
# Calculate Correlation
result <- cor(x, y)
# Display Result
print("Correlation Coefficient:")
print(result)

# Draw Scatter Plot
plot(x, y,
     main = "Correlation Analysis",
     xlab = "X Values",
     ylab = "Y Values",
     col = "blue",
     pch = 19)

# Add Regression Line
abline(lm(y ~ x), col = "red")
```

Output screenshot:



EXPERIMENT-18

SCATTER PLOT:

To draw the scatter plot using R-tool

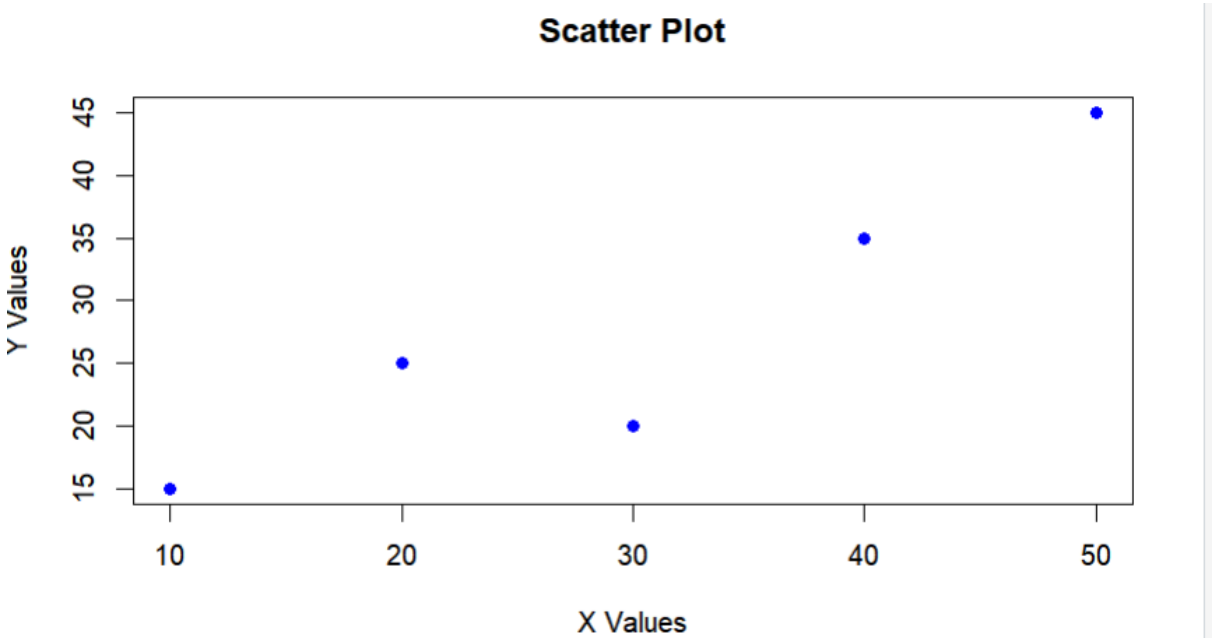
PROGRAM:

```
# Scatter Plot

x <- c(10, 20, 30, 40, 50)
y <- c(15, 25, 20, 35, 45)

plot(x, y,
     main = "Scatter Plot",
     xlab = "X Values",
     ylab = "Y Values",
     col = "blue",
     pch = 19)
```

OUTPUT SCREENSHOT:



EXPERIMENT-19

LINEAR REGRESSION:

To write thr program for the linear regression using R-tool.

PROGRAM:

```
# Linear Regression

# Create Data
x <- c(1, 2, 3, 4, 5)
y <- c(2, 4, 5, 4, 5)

# Create Data Frame
data <- data.frame(x, y)

# Apply Linear Regression
model <- lm(y ~ x, data = data)

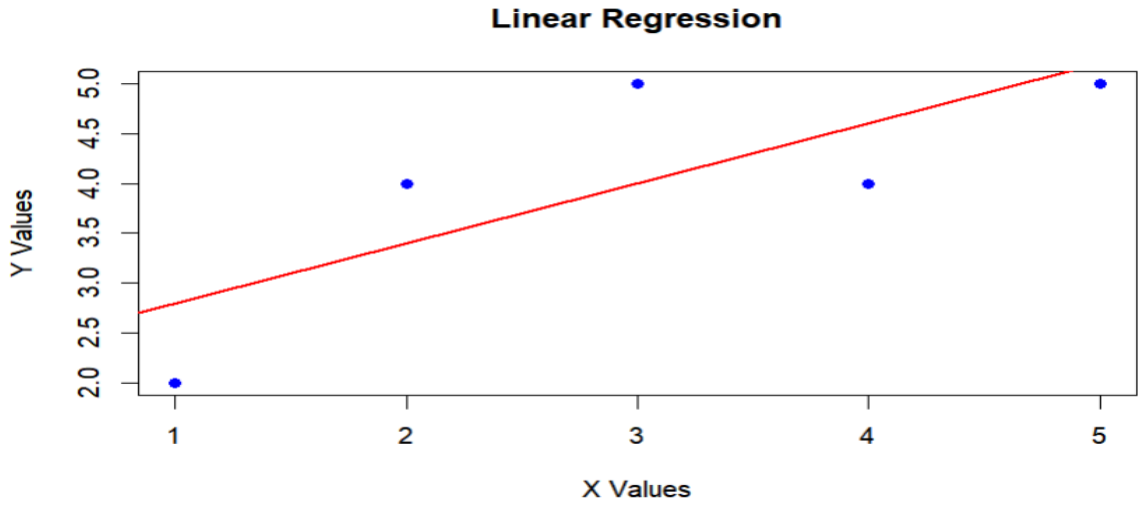
# Display Model Summary
print(summary(model))

# Predict Values
predicted <- predict(model)
print("Predicted Values:")
print(round(predicted, 2))
```

```
# Draw Scatter Plot
plot(x, y,
     main = "Linear Regression",
     xlab = "X Values",
     ylab = "Y Values",
     col = "blue",
     pch = 19)

# Add Regression Line
abline(model, col = "red", lwd = 2)
```

OUTPUT SCREENSHOT:



EXPERIMENT-20

MULTIPLE REGRESSION:

To write the program for the multiple regression.

PROGRAM:

```
# Multiple Regression

# Create Data
study_hours <- c(2, 3, 4, 5, 6)
attendance <- c(60, 65, 70, 80, 90)
marks <- c(50, 55, 65, 75, 85)

# Create Data Frame
data <- data.frame(study_hours, attendance, marks)

# Apply Multiple Regression
```

```
model <- lm(marks ~ study_hours + attendance, data = data)

# Display Model Summary
print(summary(model))

# Predict Marks
new_data <- data.frame(study_hours = 7, attendance = 95)
predicted_marks <- predict(model, newdata = new_data)

print("Predicted Marks:")
print(round(predicted_marks, 2))
```

OUTPUT SCREENSHOT:

